

Java Collections Framework

Laboratory Observation & Practical Execution Report

Course: Advanced Java Programming

Topic: Collections & Utility API

Document Type: Lab Documentation

1. System Requirements

Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher
- **Memory (RAM):** Minimum 4 GB (8 GB recommended)
- **Disk Space:** At least 500 MB free for JDK & workspace
- **Architecture:** x86_64 / ARM64 supported

Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **JDK Version:** Java Development Kit (JDK) 8 or higher
- **Environment:** JAVA_HOME environment variable configured
- **IDE/Editor:** Eclipse, IntelliJ IDEA, VS Code, or Terminal

2. Compilation & Execution Procedure

Follow these standard commands in the Terminal or Command Prompt to run Java applications:

Step 1: Open Terminal / Command Line

Navigate to the source code folder where the `.java` files are stored.

Step 2: Compilation Command

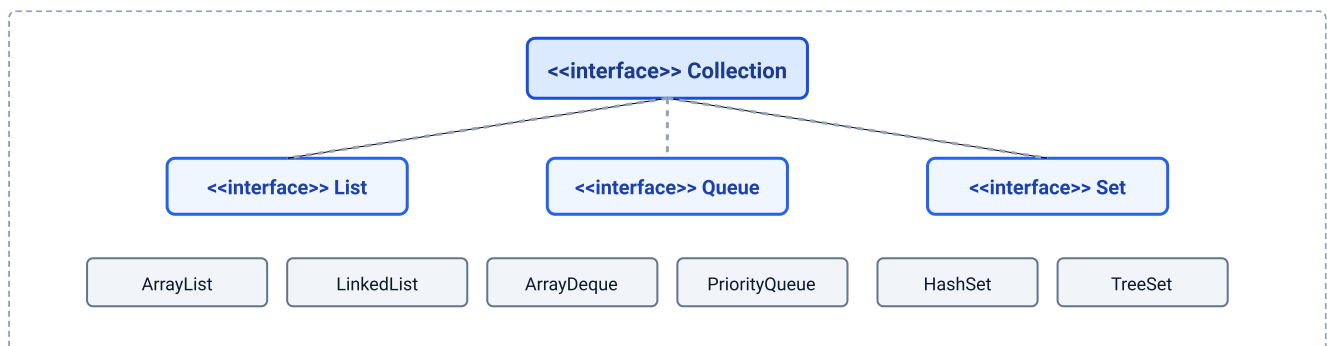
```
javac ClassName.java
```

Note: This produces a `ClassName.class` bytecode file.

Step 3: Execution Command

```
java ClassName
```

3. Collection Framework Class Diagram Overview



Objective: Perform element insertion, index-based positional operations, size evaluation, element checks, and clearing using `ArrayList`.

Source Code:

```
import java.util.ArrayList;

class ListOfArrays {
    public static void main (String[] args) {
        ArrayList<Integer> a = new ArrayList<>();
        a.add(1); a.add(2); a.add(3);
        System.out.println(a);

        a.add(3,10); a.add(4,20); a.add(5,50);
        System.out.println(a);
        System.out.println(a.size());

        ArrayList<Integer> b = new ArrayList<>();
        b.add(40); b.add(50); b.add(2,30); b.add(3,40); b.add(4,60);
        System.out.println(b);

        b.get(4);
        System.out.println(b);
        b.add(60); b.add(80);
        System.out.println(b.size());

        b.set(3, 50);
        b.remove(1);
        System.out.println(b);

        ArrayList<String> s = new ArrayList<>();
        s.add("Hello hemmanth");
        System.out.println(s);
        s.contains("Hello");
        System.out.println(s);
        System.out.println(s.contains("1234"));
        System.out.println(s.size());
        System.out.println(s.isEmpty());
        s.clear();
        System.out.println(s);
        System.out.println(s.lastIndexOf(s));
    }
}
```

Algorithm / Logic Steps:

1. Initialize integer dynamic arrays `a` and `b`.
2. Append and insert elements at specified array positions.
3. Modify elements via `set()` and remove by index via `remove()`.
4. Test search containment & inspect list emptiness after `clear()`.

Output:

```
[1, 2, 3] [1, 2, 3, 10, 20, 50] 6
[40, 50, 30, 40, 60] [40, 50, 30, 40, 60] 7
[40, 30, 50, 60, 60, 80] [Hello hemmanth]
[Hello hemmanth] false 1 false [] -1
```

Objective: Demonstrate positional inserts, sequential Deque features (`addFirst`, `poll`, `peek`) with `LinkedList`.

Source Code:

Program 2 Output & Execution Results

```
[110, 120, 130] [100, 110, 120, 130, 200] [100, 110, 120, 140, 120, 130, 200] Element at
index 2: 120 After setting index 2 to 30: [100, 110, 30, 140, 120, 130, 200] After removing
element at index 2: [100, 110, 140, 120, 130, 200] After removing first and last elements:
[110, 140, 120, 130] After removing first element: [110, 140, 120, 130] List contains 30: false
After offering 250 to the list: [110, 140, 120, 130, 250] Size of the LinkedList: 5 Is the
LinkedList empty? 110 After polling the first element: 140 After peeking the first element: 250
After clearing the LinkedList: []
```

Program 3: ListOfStacks.java

Topic: LIFO Stack Architecture

Objective: Explore LIFO (Last-In-First-Out) operations using `Stack` methods: push, pop, peek, and 1-based search.

Source Code:

```
import java.util.Stack;

public class ListOfStacks {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(110);
        stack.push(120);
        stack.push(130);
        System.out.println(stack);
        stack.pop();
        System.out.println(stack);
        stack.peek();
        System.out.println(stack);
        stack.empty();
        System.out.println(stack);
        System.out.println(stack.search(120
0));
        System.out.println(stack.search(13
0));
    }
}
```

Output Trace:

```
[110, 120, 130] [110, 120] [110,
120] [110, 120] -1 -1
```

Note: `search(130)` returns -1 as element 130 was previously popped.

Program 4: ListOfVectors.java

Topic: Synchronized Dynamic Array (Vector)

Objective: Utilize thread-safe legacy `Vector` class methods including `addElement`, capacity checks, and element query methods.

Source Code:

```
import java.util.Vector;

public class ListOfVectors {
    public static void main(String[] args) {
        Vector<Integer> vector = new Vector<>();
        vector.add(110); vector.add(120); vector.add(130);
        System.out.println(vector);
        vector.addElement(140); vector.addElement(150);
        System.out.println(vector);
        vector.get(3);
        System.out.println(vector);
        vector.set(2, 130);
        System.out.println(vector);
        vector.remove(1);
```

Program 4 Output

```
[110, 120, 130] [110, 120, 130, 140, 150] [110, 120, 130, 140, 150] [110, 120, 130, 140, 150] [110, 130, 140, 150] [110, 140, 150] 3 false true 1 10 110 150 [110, 140, 150] false
```

Program 5: ArrayDequeProgram.java

Topic: Resizable Array Deque

Objective: Perform double-ended queue actions (head/tail insertions & removals) using high-performance `ArrayDeque`.

Source Code:

```
import java.util.Deque;
import java.util.ArrayDeque;

public class ArrayDequeProgram {
    public static void main(String[] args) {
        Deque<Integer> deque = new ArrayDeque<>();
        deque.add(110); deque.add(120); deque.add(130);
        System.out.println(deque);
        deque.addFirst(100); deque.addLast(500);
        System.out.println(deque);
        System.out.println(deque.removeFirst());
        System.out.println(deque.removeLast());
        System.out.println(deque);
        System.out.println(deque.peekFirst());
        System.out.println(deque.peekLast());
        System.out.println(deque.offerFirst(90));
        System.out.println(deque.offerLast(1000));
        System.out.println(deque);
        System.out.println(deque.peekFirst());
        System.out.println(deque.peekLast());
        System.out.println(deque.pollFirst());
        System.out.println(deque.pollLast());
        System.out.println(deque);
        System.out.println(deque.peek());
        System.out.println(deque.isEmpty());
        deque.clear();
        System.out.println(deque);
    }
}
```

Output Trace:

```
[110, 120, 130] [100, 110, 120, 130, 500] 100 500 [110, 120, 130] 110 130 true
true [90, 110, 120, 130, 1000] 90 1000 90
1000 [110, 120, 130] 110 false []
```

Program 6: DequeInterface.java

Topic: Deque via LinkedList Implementation

Objective: Implement double-ended queue operations using `LinkedList` referencing the `Deque` interface.

Program 6 Code & Execution Output

```
import java.util.Deque;
import java.util.LinkedList;

public class DequeInterface {
    public static void main(String[] args) {
        Deque<Integer> deque = new LinkedList
<>();
        deque.add(110); deque.add(120); deque.add(130);
        System.out.println(deque);
        deque.offer(400);
        System.out.println(deque);
        deque.addFirst(100);
        System.out.println(deque);
        System.out.println(deque.offerFirst(90));
        System.out.println(deque.offerLast(1000));
        deque.addLast(500);
        System.out.println(deque);
        System.out.println(deque.peekFirst());
        System.out.println(deque.peekLast());
        deque.pollFirst();
        System.out.println(deque);
        deque.pollLast();
        System.out.println(deque);
        deque.removeFirst();
        System.out.println(deque);
        deque.removeLast();
        System.out.println(deque);
    }
}
```

```
[110, 120, 130] [110, 120, 130, 400]
[100, 110, 120, 130, 400] true true [90, 100,
110, 120, 130, 400, 1000, 500] 90 500 [100,
110, 120, 130, 400, 1000, 500] [100, 110,
120, 130, 400, 1000] [110, 120, 130, 400,
1000] [110, 120, 130, 400]
```

Program 7: PriorityQueueProgram.java

Topic: Min-Heap Queue

Objective: Process elements in priority order (natural ordering via Min-Heap structure) using `PriorityQueue`.

Source Code:

```
import java.util.PriorityQueue;

public class PriorityQueueProgram {
    public static void main(String[] args) {
        PriorityQueue<Integer> priorityQueue
= new PriorityQueue<>();
        priorityQueue.add(100);
        priorityQueue.add(200);
        priorityQueue.offer(130);
        priorityQueue.offer(400);
        System.out.println(priorityQueue);
        System.out.println(priorityQueue.peek());
        System.out.println(priorityQueue.poll());
        System.out.println(priorityQueue);
        priorityQueue.remove();
        System.out.println(priorityQueue);
        System.out.println(priorityQueue.contains(130));
        System.out.println(priorityQueue.size());
    }
}
```

Output Trace:

```
[100, 200, 130, 400] 100 100 [130,
200, 400] [200, 400] false 2 false []
```

Program 8 Code & Execution Output

```
import java.util.Queue;
import java.util.LinkedList;

public class QueuesOfCollection {
    public static void main(String[] args) {
        Queue<Integer> queue = new LinkedList
<>();
        queue.add(110); queue.add(120); queue.add(130);
        System.out.println(queue);
        queue.offer(400);
        System.out.println(queue);
        queue.poll();
        System.out.println(queue);
        queue.remove();
        System.out.println(queue);
        System.out.println(queue.element());
        queue.peek();
        System.out.println(queue);
        queue.isEmpty();
        System.out.println(queue);
        System.out.println(queue.contains(120));
        queue.clear();
        System.out.println(queue);
    }
}
```

```
[110, 120, 130] [110, 120, 130, 400]
[120, 130, 400] [130, 400] 130 [130, 400]
[130, 400] false []
```

Program 9: LinkedHashSetProgram.java

Topic: Ordered Unique Elements (LinkedHashSet)

Objective: Preserve element insertion ordering while ensuring uniqueness using `LinkedHashSet`.

Source Code:

```
import java.util.Set;
import java.util.LinkedHashSet;

public class LinkedHashSetProgram {
    public static void main(String[] args) {
        Set<Integer> linkedHashSet = new LinkedHashSet<>();
        linkedHashSet.add(100);
        linkedHashSet.add(200);
        linkedHashSet.add(300);
        System.out.println("LinkedHashSet: " + linkedHashSet);

        linkedHashSet.remove(200);
        System.out.println("LinkedHashSet after removing 200: " + linkedHashSet);
        System.out.println("Contains 100? " + linkedHashSet.contains(100));
        System.out.println("Size of the LinkedHashSet: " + linkedHashSet.size());
        System.out.println("Is the LinkedHashSet empty? " + linkedHashSet.isEmpty());
        linkedHashSet.clear();
        System.out.println("LinkedHashSet after clearing: " + linkedHashSet);
    }
}
```

Output Trace:

```
LinkedHashSet: [100, 200, 300]
LinkedHashSet after removing 200: [100, 300]
Contains 100? true Size of the LinkedHashSet: 2
Is the LinkedHashSet empty? false
LinkedHashSet after clearing: []
```

Program 10 Code & Execution Output

Source Code:

```
import java.util.NavigableSet;
import java.util.TreeSet;

public class NavigableSetProgram {
    public static void main(String[] args) {
        NavigableSet<Integer> navigableSet = new TreeSet<>();
        navigableSet.add(100); navigableSet.add(200); navigableSet.add(300);

        System.out.println("NavigableSet: " + navigableSet);
        System.out.println("First element: " + navigableSet.first());
        System.out.println("Last element: " + navigableSet.last());

        System.out.println("Lower than 200: " + navigableSet.lower(200));
        System.out.println("Higher than 200: " + navigableSet.higher(200));
        System.out.println("Floor of 200: " + navigableSet.floor(200));
        System.out.println("Ceiling of 200: " + navigableSet.ceiling(200));

        System.out.println("After polling first element: " + navigableSet.pollFirst());
        System.out.println("After polling last element: " + navigableSet.pollLast());
        System.out.println("Descending Set: " + navigableSet.descendingSet());

        NavigableSet<Integer> headSet = navigableSet.headSet(200, true);
        System.out.println("HeadSet (<= 200): " + headSet);
        NavigableSet<Integer> tailSet = navigableSet.tailSet(200, false);
        System.out.println("TailSet (> 200): " + tailSet);
        NavigableSet<Integer> subSet = navigableSet.subSet(100, true, 300, false);
        System.out.println("SubSet (100 to 300): " + subSet);

        navigableSet.remove(200);
        System.out.println("NavigableSet after removing 200: " + navigableSet);
        System.out.println("Contains 100? " + navigableSet.contains(100));
        System.out.println("Size of the NavigableSet: " + navigableSet.size());
        System.out.println("Is the NavigableSet empty? " + navigableSet.isEmpty());

        navigableSet.clear();
        System.out.println("NavigableSet after clearing: " + navigableSet);
    }
}
```

Execution Output

```
NavigableSet: [100, 200, 300] First element: 100 Last element: 300 Lower than 200:
100 Higher than 200: 300 Floor of 200: 200 Ceiling of 200: 200 After polling first element:
100 After polling last element: 300 Descending Set: [200] HeadSet (<= 200): [200] TailSet
(> 200): [] SubSet (100 to 300): [200] NavigableSet after removing 200: [] Contains 100?
false Size of the NavigableSet: 0 Is the NavigableSet empty? true NavigableSet after
clearing: []
```

Program 11: SetInterface.java

Topic: Basic Hashing Set (HashSet)

Objective: Demonstrate unique set storage using unordered `HashSet` implementation.

Program 11 Code & Output

```
import java.util.Set;
import java.util.HashSet;

public class SetInterface {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
        set.add("Apple");
        set.add("Banana");
        set.add("Orange");

        System.out.println("Set: " + set);
        set.remove("Banana");
        System.out.println("Set after removing Banana: " + set);

        System.out.println("Contains Apple? " + set.contains("Apple"));
        System.out.println("Size of the set: " + set.size());
        System.out.println("Is the set empty? " + set.isEmpty());

        set.clear();
        System.out.println("Set after clearing: " + set);
    }
}
```

```
Set: [Apple, Orange, Banana] Set
after removing Banana: [Apple, Orange]
Contains Apple? true Size of the set: 2 Is
the set empty? false Set after clearing: []
```

Program 12: SortedSetProgram.java

Topic: Sorted Set Interface

Objective: Access ranged subset views (`headSet` , `tailSet` , `subSet`) on sorted data structures via `SortedSet` .

Source Code:

```
import java.util.SortedSet;
import java.util.TreeSet;

public class SortedSetProgram {
    public static void main(String[] args) {
        SortedSet<Integer> sortedSet = new TreeSet<>();
        sortedSet.add(100);
        sortedSet.add(200);
        sortedSet.add(300);

        System.out.println("SortedSet: " + sortedSet);
        System.out.println("First element: " + sortedSet.first());
        System.out.println("Last element: " + sortedSet.last());
        System.out.println("HeadSet (< 200): " + sortedSet.headSet(200));
        System.out.println("TailSet (>= 200): " + sortedSet.tailSet(200));
        System.out.println("SubSet (100 to 300): " + sortedSet.subSet(100, 300));
        System.out.println("Comparator: " + sortedSet.comparator());
    }
}
```

Output Trace:

```
SortedSet: [100, 200, 300] First
element: 100 Last element: 300 HeadSet
(elements less than 200): [100] TailSet
(elements >= 200): [200, 300] SubSet
(elements between 100 and 300): [100, 200]
Comparator: null SortedSet after removing
200: [100, 300] Contains 100? true Size of
the SortedSet: 2 Is the SortedSet empty?
false SortedSet after clearing: []
```

Program 13 Code & Output

```
import java.util.Set;
import java.util.TreeSet;

public class TreeSetProgramSet {
    public static void main(String[] args) {
        Set<Integer> treeSet = new TreeSet<>
        ();
        treeSet.add(100); treeSet.add(200); treeSet.add(300);
        System.out.println("TreeSet: " + treeSet);

        treeSet.remove(200);
        System.out.println("TreeSet after removing 200: " + treeSet);
        System.out.println("Contains 100? " + treeSet.contains(100));
        System.out.println("Size of the TreeSet: " + treeSet.size());
        System.out.println("Is the TreeSet empty? " + treeSet.isEmpty());
        treeSet.clear();
        System.out.println("TreeSet after clearing: " + treeSet);
    }
}
```

```
TreeSet: [100, 200, 300] TreeSet
after removing 200: [100, 300] Contains 100?
true Size of the TreeSet: 2 Is the TreeSet
empty? false TreeSet after clearing: []
```

Program 14: CursorIterator.java

Topic: Iterator Traversal & Modification

Objective: Safely traverse and remove elements sequentially from a collection using the `Iterator` cursor.

Source Code:

```
import java.util.Iterator;

public class CursorIterator {
    public static void main(String[] args) {
        String[] arr = {"c programming", "java programming",
            "python programming",
            "c++ programming",
            "data structures"};
        Iterator<String> iterator = java.util.Arrays.asList(arr).iterator();
        while (iterator.hasNext()) {
            String element = iterator.next();
            System.out.println(element);
        }
    }
}
```

Output Trace:

```
c programming java programming
python programming c++ programming data
structures
```

Program 15: EnumProm.java

Topic: Enumeration Type with Custom Fields

Objective: Define type-safe enumerations with fields, constructors, and iteration via `values()`.

Program 15 Code & Execution Output

```
public enum EnumProm {
    JAVA("Java Programming"),
    C("C Programming"),
    CPP("C++ Programming"),
    PYTHON("Python Programming"),
    DATA_STRUCTURES("Data Structures");

    private final String description;

    EnumProm(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }

    public static void main(String[] args) {
        for (EnumProm language : EnumProm.values()) {
            System.out.println(language + ": " + language.getDescription());
        }
    }
}
```

```
JAVA: Java Programming C: C
Programming CPP: C++ Programming PYTHON:
Python Programming DATA_STRUCTURES: Data
Structures
```

Program 16: ListIteratorProm.java

Topic: Bidirectional List Iterator

Objective: Perform forward/backward traversal and dynamic inline element replacement using `ListIterator`.

Source Code:

```
import java.util.ListIterator;
import java.util.ArrayList;
import java.util.List;

public class ListIteratorProm {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple"); list.add("Banana"); list.add("Cherry");

        ListIterator<String> litr = list.listIterator();
        System.out.println("--- Forward Traversal ---");
        while (litr.hasNext()) {
            String element = litr.next();
            System.out.println("Index: " + litr.previousIndex() + " Value: " + element);
            if (element.equals("Banana")) {
                litr.set("Blueberry");
            }
        }

        System.out.println("\n--- Backward Traversal ---");
        while (litr.hasPrevious()) {
            System.out.println("Index: " + litr.previousIndex() + " Value: " + litr.previous());
        }
    }
}
```

Output Trace:

```
--- Forward Traversal --- Index: 0
Value: Apple Index: 1 Value: Banana Index: 2
Value: Cherry --- Backward Traversal ---
Index: 2 Value: Cherry Index: 1 Value:
Blueberry Index: 0 Value: Apple [Blueberry,
Cherry]
```